Dado este código:

```
package solid.exercise;
import java.util.ArrayList;
import java.util.List;

class Student {
    private String name;
    private String email;
    private double grade;

    public Student(String name, String email, double grade) {
        this.name = name;
        this.email = email;
        this.grade = grade;
    }
    public double getGrade() { return grade; }
    public String getEmail() { return email; }
    public String getName() { return name; }
}

class StudentRepository {
    private List<Student> students = new ArrayList<>();

    public void save(Student s) {
        students.add(s);
        System.out.println("Student saved: " + s.getName());
    }

    public List<Student> findAll() { return students; }
}

class GradeService {
    private StudentRepository repository = new StudentRepository();

    public void addStudent(Student student) {
        repository.save(student);
```

```
        sendEmail(student);
    }

    private void sendEmail(Student student) {
        System.out.println("Sending email to: " + student.getEmail());
    }

    public double calculateAverage() {
        List<Student> students = repository.findAll();
        double sum = 0;
        for (Student s : students) {
            sum += s.getGrade();
        }
        return sum / students.size();
    }
}

public class Main {
    public static void main(String[] args) {
        GradeService service = new GradeService();
        service.addStudent(new Student("Ana", "ana@mail.com", 8.5));
        service.addStudent(new Student("Luis", "luis@mail.com", 9.0));
        System.out.println("Average grade: " + service.calculateAverage());
    }
}
```

Responde a las siguientes preguntas:

1. ¿Qué principio SOLID se viola al tener el envío de correos dentro de `GradeService`?
2. ¿Qué principio se incumple al crear `StudentRepository` directamente dentro de `GradeService`?
3. ¿Qué principio garantiza que cada clase tenga una sola responsabilidad?
4. ¿Por qué el código actual sería difícil de extender sin modificar clases existentes?
5. ¿Qué beneficio tendría aplicar la Inversión de Dependencias en este ejemplo?

1. **Respuesta:** Se viola el **SRP (Single Responsibility Principle)**, porque `GradeService` tiene más de una responsabilidad: gestionar notas y enviar correos.
2. **Respuesta:** Se incumple el **DIP (Dependency Inversion Principle)**, porque `GradeService` depende directamente de una implementación concreta (`StudentRepository`) en lugar de depender de una abstracción (una interfaz).
3. **Respuesta:** El **SRP (Single Responsibility Principle)**.
4. **Respuesta:** Porque **no sigue el OCP (Open/Closed Principle)**; las clases no están abiertas a extensión y cualquier cambio (por ejemplo, otro tipo de almacenamiento o notificación) requeriría modificar las clases existentes.
5. **Respuesta:** Permitiría **cambiar la implementación de almacenamiento o notificación sin modificar `GradeService`**, aumentando la flexibilidad y facilitando pruebas unitarias.